

Part IV: Greedy Algorithms

Lecture 13: Huffman Coding

Computer Science and Technology
United International College

Objective and Outline

Objective: Another example of greedy algorithms

Reference: Section 16.3 of CLRS

Outline:

- 1 Coding and Decoding
- 2 The optimal source coding problem
- 3 Huffman coding: A greedy algorithm
- 4 Correctness

Example

Suppose that we have a 100, 000 character data file that we wish to store. The file contains only 6 characters, appearing with the following frequencies:

	a	b	c	d	e	f
Frequency in '000s	45	13	12	16	9	5

A **binary code** encodes each character as a binary string or **codeword**.

- a **code** is a set of codewords
- e.g., {000, 001, 010, 011, 100, 101} and {0, 101, 100, 111, 1101, 1100}

Given a code (corresponding to some **alphabet** Σ) and a message it is easy to **encode** the message. Just replace the characters by the codewords.

Example

$$\Sigma = \{a, b, c, d\}$$

If the code is

$$C_1 = \{a = 00, b = 01, c = 10, d = 11\}$$

then **bad** is encoded into **01 00 11**

If the code is

$$C_2 = \{a = 0, b = 110, c = 10, d = 111\}$$

then **bad** is encoded into **110 0 111**

Fixed-Length vs Variable-Length

In a **fixed-length code** each codeword has the **same** length.

In a **variable-length code** codewords may have **different** lengths.

Example

	a	b	c	d	e	f
Frequency in '000s	45	13	12	16	9	5
fixed-len code	000	001	010	011	100	101
variable-len code	0	101	100	111	1101	1100

(note that, since there are 6 characters, a fixed-length code must have at least 3 bits per codeword).

- The fixed-length code requires 300,000 bits to store the file.
- The variable-length code uses only
 $(45 * 1 + 13 * 3 + 12 * 3 + 16 * 3 + 9 * 4 + 5 * 4) * 1000 = 224,000$ bits, saving a lot of space!

$$C_1 = \{a = 00, b = 01, c = 10, d = 11\}$$

$$C_2 = \{a = 0, b = 110, c = 10, d = 111\}$$

$$C_3 = \{a = 1, b = 110, c = 10, d = 111\}$$

Given an encoded message, **decoding** is the process of turning it back into the original message.

A message is **uniquely decodable** (vis-a-vis a particular code) if it can only be decoded in one way.

Example

Relative to C_1 , **010011** is uniquely decodable to **bad**.

Relative to C_2 , **1100111** is uniquely decodable to **bad**. But,

relative to C_3 , **1101111** is not uniquely decodable since it could have encoded either **bad** or **acad**.

In fact, one can show that every message encoded using C_1 or C_2 is uniquely decipherable. The unique decipherability property is needed in order for a code to be useful.

Fixed-length codes are always uniquely decipherable

Question

Why?

We saw before that these do not always give the best **compression** so we prefer to use variable length codes.

Definition

A code is called a **prefix (free) code** if no codeword is a prefix of another one.

Example

$\{a = 0, b = 110, c = 10, d = 111\}$ is a prefix code.

Important Fact: Every message encoded by a prefix free code is uniquely decipherable.

- Since no codeword is a prefix of any other, we can always find the first codeword in a message, peel it off, and continue decoding.

Example

code: $\{a = 0, b = 110, c = 10, d = 111\}$.

$$01101100 = 01101100 = abba$$

We are therefore interested in finding good (best compression) prefix-free codes.

Outline

- 1 Coding and Decoding
- 2 The optimal source coding problem
- 3 Huffman coding: A greedy algorithm
- 4 Correctness

The Optimal Source Coding Problem

Problem

Given an alphabet $A = \{a_1, \dots, a_n\}$ with frequency distribution $f(a_i)$, find a binary prefix code C for A that minimizes the number of bits

$$B(C) = \sum_{i=1}^n f(a_i) L(c(a_i))$$

needed to encode a message of $\sum_{i=1}^n f(a_i)$ characters, where

- $c(a_i)$ is the codeword for encoding a_i and
- $L(c(a_i))$ is the length of the codeword $c(a_i)$

Example

Problem

Suppose we want to store messages made up of 4 characters a, b, c, d with frequencies 60, 5, 30, 5 (percents) respectively. What are the fixed-length codes and prefix-free codes that use the least space?

Solution:

characters	a	b	c	d
frequency	60	5	30	5
fixed-length code	00	01	10	11
prefix code	0	110	10	111

To store 100 of these characters,

(1) the fixed-length code requires $100 * 2 = 200$ bits,

(2) the prefix code uses only

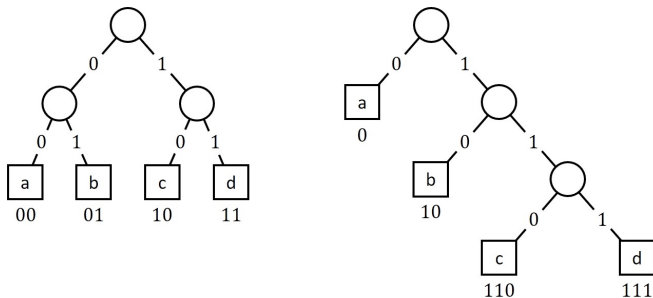
$$60 * 1 + 5 * 3 + 30 * 2 + 5 * 3 = 150$$

a 25% saving.

Remark: We will see later that this is the optimum (lowest cost) prefixcode.

Correspondence between Binary Trees and Prefix Codes

1-1 correspondence between **leaves** and **characters**.



- Left edge is labeled 0; right edge is labeled 1
- The binary string on a **path from the root to a leaf** is the **codeword** associated with the character at the leaf.

Minimum-Weight External Pathlength Problem

- $d(a_i)$, the depth of leaf a_i , is equal to the length $L(c(a_i))$ of the codeword associated with that leaf.
- **Weighted external pathlength** $B(T)$ of tree T

$$\sum_{i=1}^n f(a_i) d(a_i) = \sum_{i=1}^n f(a_i) L(c(a_i))$$

Definition (Minimum-Weight External Pathlength Problem)

Given weights $f(a_1), \dots, f(a_n)$, find a tree T with n leaves labeled $a_1, \dots, a_n$ that has **minimum** weighted external path length.

The Huffman encoding problem is **equivalent** to the minimum-weight external pathlength problem.

Outline

- 1 Coding and Decoding
- 2 The optimal source coding problem
- 3 Huffman coding: A greedy algorithm
- 4 Correctness

Huffman Coding

① (Greedy idea)

- Pick two characters x, y from alphabet A with the **smallest** frequencies
- create a subtree that has these two characters as leaves.
- Label the root of this subtree as z .

②

- Set frequency $f(z) = f(x) + f(y)$
- Remove x, y and add z creating a new alphabet A'
 - $A' = A \cup \{z\} - \{x, y\}$
 - Note that $|A'| = |A| - 1$

Repeat this procedure, called **merge**, with the new alphabet A' until an alphabet with only one symbol is left.

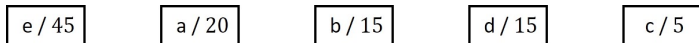
The resulting tree is the Huffman code

- the **optimum** (minimum-cost) prefix code for the given frequency distribution.

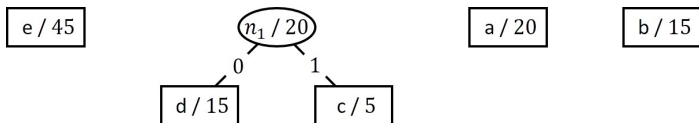
Example of Huffman Coding I

Let $A = \{e/45, a/20, b/15, d/15, c/5\}$ be the alphabet and its frequency distribution. Elements in A are sorted by their frequency.

0 Initialization



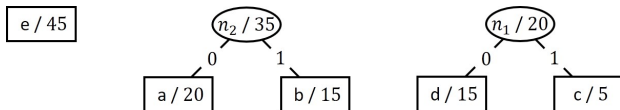
1 The first step of Huffman coding merges c and d (could also have merged c and b).



Alphabet is now $A_1 = \{e/45, n_1/20, a/20, b/15\}$.

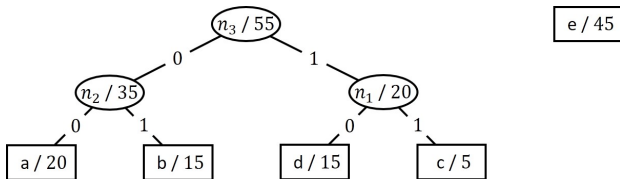
Example of Huffman Coding II

- 2 Algorithm merges a and b (could also have merged n_1 and b)



New alphabet is $A_2 = \{e/45, n_2/35, n_1/20\}$.

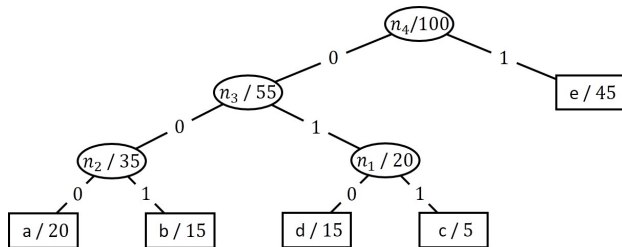
- 3 Algorithm merges n_1 and n_2 .



New alphabet is $A_3 = \{n_3/55, e/45\}$.

Example of Huffman Coding III

- 4 Algorithm merges e and n_3 and finishes.



The Huffman code is:

$a = 000, b = 001, c = 010, d = 011, e = 1.$

Huffman Coding Algorithm

Given an alphabet A with frequency distribution $\{f(a) : a \in A\}$. The binary Huffman tree is constructed using a **priority queue**, Q , of nodes, with frequencies as keys.

Algorithm 1 Huffman(A)

```
1:  $n = |A|$ 
2:  $Q = A$     // the future leaves
3: for all  $i = 1$  to  $n - 1$  do    // Why  $n-1$ ?
4:      $z = \text{new node}$ 
5:      $\text{left}[z] = \text{Extract-Min}(Q)$ 
6:      $\text{right}[z] = \text{Extract-Min}(Q)$ 
7:      $f[z] = f[\text{left}[z]] + f[\text{right}[z]]$ 
8:      $\text{Insert}(Q, z)$ 
9: end for
10: return  $\text{Extract-Min}(Q)$     // root of the tree
```

Running time is $\mathcal{O}(n \log n)$, as each priority queue operation takes time $\mathcal{O}(\log n)$.

Outline

- 1 Coding and Decoding
- 2 The optimal source coding problem
- 3 Huffman coding: A greedy algorithm
- 4 Correctness

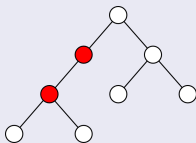
Lemma 1

Lemma (1)

An **optimal prefix code** tree must be "full", i.e., that every internal node has exactly two children.

Proof.

If some internal node had only one child,



then we could simply get rid of this node and replace it with its unique child. This would decrease the total cost of the encoding. □

Lemma (2)

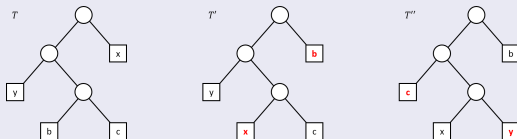
Let T be **prefix code** tree and T' be another obtained from T by swapping two leaf nodes x and b . If

$$f(x) \leq f(b), d(x) \leq d(b)$$

then,

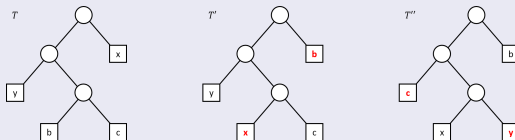
$$B(T') \leq B(T)$$

Moving a small-frequency character downward in T doesn't make it worse.



Lemma 2 II

Proof.



$$\begin{aligned} B(T') &= B(T) - f(x)d(x) - f(b)d(b) + f(x)d(b) + f(b)d(x) \\ &= B(T) + \underbrace{(f(x) - f(b))}_{\geq 0} \underbrace{(d(b) - d(x))}_{\geq 0} \\ &\leq B(T) \end{aligned}$$



Lemma (3)

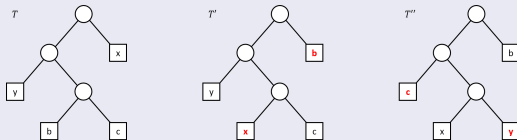
Consider the two characters x and y with the **smallest frequencies**. There is an optimal code tree in which these two letters are **sibling** leaves at the **deepest** level of the tree.

Lemma 3 II

Proof.

Let

- T be any optimal prefix code tree,
- b and c be two siblings at the deepest level of the tree (must exist because T is full).



Assume without loss of generality that $f(x) \leq f(b)$ and $f(y) \leq f(c)$

- (If necessary) swap x with b and swap y with c .
- Proved due to Lemma 2.



Lemma 4

Lemma (4)

- Let T be a prefix code tree and x and y are two sibling leaves.
- Let T' be obtained from T by removing x and y , naming the parent z , and setting $f(z) = f(x) + f(y)$
- Then

$$B(T) = B(T') + f(x) + f(y)$$

Proof.

$$\begin{aligned} B(T) &= B(T') - f(z)d(z) + f(x)d(x) + f(y)d(y) \\ &= B(T') - f(z)d(z) + f(x)(d(z) + 1) + f(y)(d(z) + 1) \\ &= B(T') - (f(x) + f(y))d(z) + (f(x) + f(y))(d(z) + 1) \\ &= B(T') + f(x) + f(y) \end{aligned}$$



Huffman Codes are Optimal I

Lemma (5)

The prefix code tree (hence the code) produced by the Huffman coding algorithm is optimal.

Proof.(By induction on n , the number of characters)

Induction basis: $n = 2$, Tree with two leaves. Obviously optimal.

Induction hypothesis: Huffman's algorithm produces optimal tree in the case of $n - 1$ characters.

Induction step: Consider the case of n characters:

- Let H be the tree produced by the Huffman's algorithm.
- Need to show: H is optimal.

Huffman Codes are Optimal II

Proof.(Con't)

- Due to the way Huffman's algorithm works,
 - There are two characters x and y with the smallest frequencies that are sibling leaves in H .
- Let H' be obtained from H by removing x and y , naming the parent z , and setting $f(z) = f(x) + f(y)$
- Alphabet for H : A ; Alphabet for $A' = A - \{x, y\} \cup \{z\}$
- By Lemma 4,

$$B(H) = B(H') + f(x) + f(y)$$

- H' is the tree produced by Huffman's algorithm for A' .
- By the induction hypothesis, H' is optimal for A' .

Huffman Codes are Optimal III

Proof.(Con't)

- By Lemma 3, there exists an optimal tree T where x and y are sibling leaves.
- Let T' be obtained from T by removing x and y , naming the parent z .
- T' is a prefix code tree for alphabet A' .
- By Lemma 4, $B(T) = B(T') + f(x) + f(y)$
- Hence

$$\begin{aligned} B(H) &= B(H') + f(x) + f(y) \\ &\leq B(T') + f(x) + f(y) \quad (H' \text{ is optimal for } A') \\ &= B(T). \end{aligned}$$

- Therefore, H must be optimal. Proved.